



P3

故障熔断治理对 Agent 矩阵收口率的因果效应：一项预注册全因子随机对照仿真

The Causal Effect of Fault-Breaker Governance on Agent Matrix Completion Rate: A Pre-Registered Full-Factorial Randomized Controlled Simulation



故障熔断治理对 Agent 矩阵收口率的因果效应：一项预注册全因子随机对照仿真

UDOS 推演引擎 v7.5.0 · 房文新 · 2026-09-20

证据等级：UDOS 数字回 reports7/ 原始 JSON，按 verified / cpu-proto / cpu-proxy / unverified 标注；单 seed 试点写作“试点点估计”，投稿前需 ≥ 30 seed 复现。

本文为 UDOS 推演引擎 v7.5.0 (main tag `v7.5.0`, commit `1a71270`) 的因果推断型论文。治理三件套（熔断/快照回滚/改派）、Owner/Trace/Stop 治理与矩阵端到端闭环均为本仓 `udos7/topology/` 的可运行实现。当前报告为单 seed 试点点估计；全因子随机对照仿真的设计、主指标、护栏与停止规则已按预注册纪律写就，多 seed 实验为投稿前待补（见第 5、8 节）。所有数字来自 `reports7/matrix_scale_demo.json`（证据等级 cpu-proto）。

结构化摘要（背景与问题 → 方法 → 证据与结果 → 贡献）

【背景与问题】 多智能体矩阵必然遇到故障（执行节点崩溃、交接丢上下文、拜占庭式错误输出）。产业演示多只报”治好了”的成功案例，既不拆”到底哪个治理机制在起作用”，也不报”治不好时系统是否如实失败”，更不报治理自身的代价（详见 § 1.1、§ 2.4）。

【方法】 把故障治理拆成可独立开关的因子：韧性三件套（三级熔断 / 快照回滚 / 改派）与责任治理（唯一 Owner / 哈希链 Trace / 显式 Stop Condition），并以预注册的全因子随机对照仿真估计”治理开关 × 故障模式”对收口率的因果效应；主指标收口率 = $\text{accepted}/\text{total}$ （工单被 QA 机械验收通过比例），护栏为返工、消息、误隔离率、Trace 断裂数、kill-switch 误触发率（详见 § 3、§ 4）。

【证据与结果】 单 seed 试点（24 工单、9 specialist、7 QA 含 2 拜占庭）：无故障 24/24 收口、0 返工；注入 crash/drop_context/byzantine 三类故障后仍 24/24 收口、3 故障节点全隔离、5 次返工、治理六项与守恒、Trace 完整性保持；而在”全领域故障 + 零重试预算”极端场景下，系统如实触发 kill-switch、报告未收口（ $\text{accepted} < 24$ 、 $\text{governance.ok} = \text{false}$ ），不冒充成功（详见 § 6）。

【贡献】 (i) 把治理机制因子化为可独立开关、可测边际贡献；(ii) 按预注册 RCT 纪律（冻结主指标/护栏/ITT 停止规则/seed 表）把因果设计写死、数据后补；(iii) 把”如实失败（kill-switch）”作为一等结果。当前为单 seed 试点点估计，全因子（ $5 \times 5 \times 5 \times 4$ ） $\times \geq 30$ seed 与统计量为待补，本文不虚构 p 值/CI（详见 § 7、§ 8）。

中文摘要

多智能体（multi-agent）矩阵在真实运行中必然遇到故障：执行节点崩溃、交接丢失上下文、甚至给出拜占庭式错误输出。问题不在于”会不会坏”，而在于”治理机制能否因果地把收口率拉回来，代价是什么”。本文把故障治理拆成三组可独立开关的机制——熔断（三级断路器）、快照回滚（哈希链 Trace 截断）、改派（在途工单转交健康节点），以及一组责任治理（唯一 Owner、哈希链 Trace、显式 Stop Condition）——并以预注册的全因子随机对照仿真，估计”治理开关 × 故障模式”对任务收口率的因果效应。本文先在 24 工单、9 名 specialist、7 名 QA（含 2 名拜占庭 QA）的小规模矩阵上给出单 seed 试点：无故障时 24/24 收口、0 返工；注入 3 个故障 specialist（crash / drop_context / byzantine 各一）后，仍 24/24 收口、3 个故障节点全部隔离、5 次返工、治理六项与守恒、Trace 完整性保持；而在”全领域故障 + 零重试预算”的极端场景下，系统如实触发 kill-switch、报告未收口（ $\text{accepted} < 24$ 、 $\text{governance.ok} = \text{false}$ ），不冒充成功。主指标预注册为收口率（ $\text{accepted}/\text{total}$ ），护栏为返工次数、消息总数、误隔离率、Trace 断裂数、kill-switch 误触发率。本文的贡献是：把故障治理因子化、预注册化，并诚实地把”能拉回收口率的机制”与”拉不回来、应如实失败的边界”同时测量出来。需要说明：上述”全领域故障 + 零重试预算触发

kill-switch”的反例为按设计构造的极端场景，当前仅在 one-pager/CHANGELOG 中定性记录，其精确 accepted 数与治理项明细尚待重跑确认（正文 6.4 节标 [RESULT NEEDED] ），不在本次 matrix_scale_demo.json 的 clean/stressed 两跑之内。

关键词：多智能体系统；故障容错；熔断；随机对照实验；预注册；收口率；拜占庭容错

English Abstract

Multi-agent matrices inevitably encounter faults: crashed executors, lost handoff context, and even Byzantine outputs. The question is not whether faults occur, but whether governance mechanisms causally recover task completion, and at what cost. This paper decomposes fault governance into independently switchable mechanisms—three-tier circuit breaking, snapshot rollback over a hash-chained Trace, and reassignment—and a responsibility layer (unique Owner, hash-chained Trace, explicit Stop Condition). A pre-registered full-factorial randomized controlled simulation estimates the causal effect of “governance switches $\times$ fault modes” on completion rate. We first report a single-seed pilot on a 24-order matrix (9 specialists, 7 QA voters including 2 Byzantine): no faults yield 24/24 completion with 0 reworks; injecting three faulty specialists (crash / drop_context / byzantine) still yields 24/24 completion, isolates all three faulty nodes, incurs 5 reworks, and keeps governance, conservation, and Trace integrity; under an extreme “all-domain fault plus zero retry budget” scenario, the system honestly trips the kill-switch and reports non-completion (accepted<24, governance.ok=false) rather than faking success. The pre-registered primary metric is completion rate; guardrails are reworks, message count, false-isolation rate, Trace breakage, and spurious kill-switch triggers. We contribute a factorized, pre-registered measurement of fault governance that reports both what recovers completion and the boundary beyond which honest failure is the correct behavior. Note: the “all-domain fault plus zero retry budget trips the kill-switch” counterexample is a designed extreme scenario currently recorded only qualitatively in a one-pager/CHANGELOG; its exact accepted count and governance details are pending a re-run (see Sec. 6.4), and it is not part of the clean/stressed runs in the current matrix_scale_demo.json.

Keywords: multi-agent systems; fault tolerance; circuit breaker; randomized controlled trial; pre-registration; completion rate; Byzantine fault tolerance

1 引言

1.1 外部产业动机（外部数字均为 unverified，报道口径，未独立核验）

随着多智能体编排系统走向生产，“故障治理”从工程配角变成核心问题。外部报道中不乏”多智能体自主完成复杂工作流”“自我修复的 Agent 系统”等叙事（此类外部说法属报道口径，未独立核验，标记为 unverified，仅作动机，不与本仓数字互证）。[CITATION NEEDED: LLM agent fault tolerance self-healing multi-agent production] 但产业界普遍缺少的，是把”治理到底在多大程度上因果地有效”做成分

层、可开关、可预注册测量的实验。大多数演示只展示”治好了”的成功案例，不报告”治不好时系统是否如实失败”，也不报告”治理本身的代价”（误隔离健康节点、额外返工、额外消息）。

本文认为，一个诚实的故障治理研究必须同时回答三个问题：(1) 哪些治理机制因果地提升了收口率？(2) 这些机制的护栏代价是什么？(3) 在什么极端场景下，正确的行为不是”硬治”，而是”如实失败”？

之所以要把”如实失败”单独提出，是因为多智能体系统有一种天然的诱惑：当系统无法完成任务时，它仍然可以输出一个”看起来完成了”的结果——也许是某个 Agent 编造了答案，也许是 QA 委员会被拜占庭多数裹挟，也许是调度器为了不让用户失望而硬凑收口。在生产系统里，这种”假成功”比”明说失败”危险得多：它把错误悄悄传递给下游，等到爆发时已经无法追溯。因此，本文把治理系统的目标定义为三件事：**能恢复时恢复、不能恢复时隔离并改派、彻底不能时如实停止并留下完整审计痕迹**。这三个目标缺一不可，只做前两个、漏掉第三个，就不是一个诚实的治理系统。

外部产业叙事（如”自我修复 Agent””自主诊断 workflow”，均为 unverified 报道口径）大多只演示前两个目标的成功案例。本文要补的，是第三个目标的测量，以及前两个目标中每个机制的边际因果贡献。

更进一步说，当前多智能体故障治理的文献有两个共性短板：一是”黑箱式有效”——只报告开了治理后成功率提升多少，不拆开到底是熔断、回滚还是改派在起作用；二是”报喜不报忧”——只展示恢复成功的案例，回避”什么时候系统应该承认自己做不到”。前者让工程师无法知道该投资哪个机制，后者让生产系统在超过恢复能力时仍输出不可信结果。本文用因子化开关解决第一个短板，用 kill-switch 反例如实报告解决第二个短板。这正是本文定位为”因果效应”而非”效果演示”的原因。

1.2 研究问题

本文聚焦三个研究问题：

- **RQ1**：唯一 Owner、哈希链 Trace、显式 Stop Condition 三要素各自的边际贡献是什么？去掉哪一个会导致什么类型的失败？
- **RQ2**：熔断/回滚/改派三件套，在 crash / drop_context 故障下，能否把收口率拉回无故障水平？阈值过低时，是否会误隔离健康节点（护栏恶化）？
- **RQ3**：byzantine 故障能否仅靠执行层熔断隔离？还是必须靠 QA 共识 + 质量失败累积才能隔离？

1.3 贡献

- **因子化治理开关**：把治理拆成可独立关闭的机制（-Owner、-Trace、-Stop、-熔断、-回滚、-改派），使边际贡献可被实验分离。
- **预注册 RCT 设计**：按随机对照实验纪律，预先冻结主指标、护栏、停止规则、随机化单元与纳入排除标准（第 4 节）。
- **含”如实失败”的试点证据**：不仅报告 24/24 收口的成功案例，也报告”全领域故障 + 零重试预算”下 kill-switch 触发、系统如实报告未收口的反例。
- **诚实标注**：当前为单 seed 试点估计，全因子多 seed 实验为待补，本文不虚构 p 值、CI 与样本量。

1.4 范围声明

本文研究的是”故障治理机制对收口率的因果效应”，不是”如何训练一个更可靠的 LLM”。Agent 的智能由外部模型提供，本文将其抽象为确定性处理器，故障由人为注入。因此本文的结论是协议层、机制层的因果效应，不涉及模型本身的可靠性提升。读者对象是做多智能体系统设计、故障容错、以及关心预注册因果实验的系统研究者。超出这一边界（真实模型可靠性、真实网络延迟）的结论在第 8 节闸门中标注为未达。

2 相关工作

2.1 微服务熔断与容错

熔断（circuit breaker）在微服务治理中成熟：closed/open/half-open 状态机、错误率阈值、冷却探测、半开恢复。[CITATION NEEDED: circuit breaker pattern microservices fault tolerance] 与”在不确定性下降级”同源的另一条线索是共形预测：**概念**上，当预测不确定时应给出覆盖保证而非强行下结论；**机制**上，Angelopoulos 与 Bates 的共形预测综述提供分布无关、有限样本的预测区间（Angelopoulos & Bates, 2021, arXiv:2107.07511）；**证据**上，它说明”不确定性高就降级/交接”是一条有统计底座的设计原则。本仓 `udos7/topology/circuit_breaker.py` 把熔断模式扩展到三级：Agent 级断路器、子矩阵级组熔断、全局 kill-switch，并增加”故障时在途工单改派”这一多智能体特有的恢复动作。

微服务熔断通常只对远程调用失败生效，而多智能体场景的故障更”语义化”：一个 Agent 可能在没有任何网络错误的情况下，给出看似成功、实则错误的结果（拜占庭），或者因为交接时丢失上下文而无法继续（drop_context）。这类故障不会触发传统的网络超时或 5xx，因此需要把”错误”的定义从”调用失败”扩展到”结果不满足验收谓词”。在具身侧，SayCan 的思路与之呼应：**概念**上，LLM 给步骤候选、技能价值函数给可行性 affordance；**机制**上，当 affordance 不满足即不执行该步（Ahn, Brohan, Brown et al., 2022, arXiv:2204.01691）；本文把”affordance 不满足即换人/换技能”泛化为”验收谓词不满足即触发熔断/改派”。本文的治理实验正是要测量：在这种语义故障下，传统熔断模式加上”结果验收驱动的质量失败累积”，能否因果地恢复收口率。

2.2 拜占庭容错与停止共识

BFT 共识是成熟机制：**概念**上，异步网络中存在任意作恶（equivocation）节点时如何仍就状态达成一致；**机制**上，Castro 与 Liskov 的 PBFT 用 pre-prepare/prepare/commit 三阶段复制，要求 $n \geq 3f+1$ 容忍 f 个拜占庭节点、需 $2f+1$ 法定人数，配合 view change（Castro & Liskov, 1999, OSDI 1999）；**证据**上，其 BFS 实现仅比未复制 NFS 慢约 3%。本仓 `udos7/topology/consensus.py` 的 `StopConsensus` 借用它来对”验收通过（stop）/返工（continue）”投票：拜占庭 QA 谎报无法对抗诚实多数。与 QA 委员会同源的另一条线索是 LLM-as-a-Judge: Zheng 等人证明强模型作为评审者与人类评委一致率 >80%，但存在位置/冗长/自我偏好偏见（Zheng, Chiang, Sheng et al., 2023, NeurIPS 2023 Datasets, arXiv:2306.05685）；这提示我们”用多个 QA 投票”本身也需防评委偏见，本试点用 7 人委员会正是为了用多数票压单点偏见。

在本试点中，QA 委员会 7 人、其中 2 名拜占庭（ $f=2$ ），满足 $n \geq 3f+1$ （ $7 \geq 7$ ）这一最严格的拜占庭容错条件。这意味着即使 2 名 QA 串通投反对票，诚实多数仍能正确决定验收。本试点中 2 名拜占庭 QA 的存在，正是为了检验这种”验收层共识能否在存在 Byzantine 时仍正确收口”——结果是 24/24 收口，说明共识设计按预期工作。需要强调，这只是协议层正确性的演示，不是对真实 LLM 投票行为的测量。

2.3 可观测性与责任追溯

哈希链追加式日志可发现篡改与断链；唯一 Owner 登记可检测重复劳动。[CITATION NEEDED: hash chained audit log ownership attribution] 本仓 `governance.py` 的 `TraceLedger` + `audit_run` 把状态丢失、重复劳动、责任不清、提前终止、无界循环五类失败机械化检测。

与一般可观测性只”展示指标”不同，本文的责任治理把”谁对哪个阶段负责”“这个结果是否可追溯”做成了硬约束：`claim(order, stage, agent)` 一旦发现同一 `order:stage` 被第二个不同 owner 认领，就计入 `duplicate_work`；`audit_run` 在运行结束时对每一类失败给出明确列表。这使得”重复劳动”“责任不清”不再是需要事后人工复盘的模糊概念，而是可在实验中被直接计数的护栏指标。

2.4 现有缺口

我们假设（待检索证实/证伪）：熔断在微服务领域成熟，但缺少在 Agent 语义故障（上下文丢失、拜占庭输出）下的因子化因果实验；也很少有研究把”治理如实失败”作为一等测量目标。概念上，AutoGen 类可对话多智能体框架已观察到”对话无终止条件易死循环、成本随轮数膨胀”这一多智能体特有的失败模式

（Wu, Bansal, Zhang et al., 2023, arXiv:2308.08155）；机制上，DeepSeek-R1 等纯 RL 路线则展示了”熔断/失败后由可验证奖励驱动恢复与自我博弈”的可能（DeepSeek-AI, 2025, arXiv:2501.12948）。证据上，这些工作证明多智能体失败与恢复是真问题，但多在框架/训练层面讨论，少有把治理开关作为因子做受控因果测量。与 UDOS 的差异在于：本文把治理机制拆成可开关的三件套，用预注册 RCT 测量其因果效应，并把”治理如实失败”作为一等测量目标。[CITATION NEEDED: chaos engineering LLM agents fault injection controlled experiment] 本文试图用预注册 RCT 填补这一缺口。

2.5 预注册与可复现因果推断

在系统与实验研究中，“看到结果后再挑指标”是最常见的效度威胁之一。预注册（pre-registration）通过在收集数据前冻结主指标、护栏、样本量与停止规则，降低这种窥探偏差。[CITATION NEEDED: pre-registration empirical software engineering experiment reproducibility] 本文把这一纪律用到故障治理实验：第 4.4 节的因子、主指标（收口率）、护栏（返工/消息/误隔离/trace 断裂）、ITT 停止规则，都在跑全因子前冻结。当前报告的单 seed 试点是”设计先行、数据后补”的第一步：先把方案写死，再逐步补数据，而不是反过来。这一立场与本仓 P12 论文的证据分级方法论一致。

2.6 混沌工程与故障注入

混沌工程主张主动向系统注入故障、观察韧性，而非等待真实事故。[CITATION NEEDED: chaos engineering fault injection resilience] 本文的故障注入（crash / drop_context / byzantine / duplicate）正是这一思路在多智能体矩阵上的小型化：不等待真实 LLM 出故障，而是在实验台上主动、可控、可复现地注入语义故障，测量治理机制的响应。区别在于，传统混沌工程多在微服务/基础设施层注入网络延迟、节点宕机，本文额外注入”语义故障”——节点看似正常却产出错误结果（byzantine），这是多智能体特有的失败模式。

3 问题定义与假设

3.1 故障模型

本文用四类人为离散故障（源码 `FAULTS` 与 `AgentSpec.fault`）：

- **crash**：执行节点直接失败（`error_type="crash"`），无输出。
- **drop_context**：执行节点丢失上下文（`error_type="missing_context"`，`note="未收到 domain/units"`），无法正确完成。
- **byzantine**：执行节点“看似成功”但输出错误（`specialist` 返回 `true_result + 7`，或 `triage` 把领域说错）。
- **duplicate**：重复劳动。

3.2 可证伪假设

- **H1（治理要素边际）**：去掉唯一 Owner → 重复劳动率上升；去掉 Trace → 状态丢失后无法 replay，收口率下降；去掉 Stop Condition → 出现无界循环或提前终止。
- **H2（三件套恢复）**：熔断 + 回滚 + 改派使 crash / drop_context 下收口率回到无故障水平；熔断阈值过低则健康节点被误隔离（护栏恶化）。
- **H3（拜占庭需共识）**：byzantine 故障无法仅靠执行层熔断隔离，必须靠 QA 共识 + 质量失败累积隔离。

可证伪判据：若去掉某治理要素后收口率无显著变化，该要素的边际贡献假设被削弱；若三件套下 crash 故障仍无法收口，H2 被推翻；若 byzantine 故障在无 QA 共识时也能被执行层隔离，H3 被削弱。

3.3 为什么用“收口率”作主指标

多智能体任务的成功难以用单一数值刻画，但本文的工单是机械可判定的：每个工单内藏真值（领域 + 单元求和），`WorkOrder.verify(result, domain)` 严格比较结果与真值、领域与真领域。因此“收口”被操作化为“该工单最终被 QA 委员会验收通过（stop）”， $\text{收口率} = \text{accepted} / \text{total}$ 。这避免了主观评分带来的评分者间不一致，使因果效应估计在确定性 CPU 环境下可复现。需要强调，选择收口率不是因为它唯一重要，而是因为它在本实验台可机械判定；返工次数、消息数、误隔离率作为护栏，刻画“为了收口付出的代价”。

4 方法与系统设计

4.1 三件套：三级熔断 + 快照回滚 + 改派

源码 `udos7/topology/circuit_breaker.py`：

- **AgentBreaker**：滑动窗口错误率，`record(ok)` 累计 success/failure，当 `successes+failures ≥ min_requests` 且 `error_rate ≥ trip_threshold` 时转 OPEN（隔离）；冷却 `cooldown_ticks` 后半开 `HALF_OPEN`，`probe(ok)` 成功才回 closed。`available` 属性只有 closed 才为真，因此熔断节点在派单时被排除。

- **GroupBreaker**: 组内 OPEN 成员比例 $\geq$ `isolate_ratio_threshold` 则整组熔断。
- **KillSwitch**: `trip(reason)` 后 `allow_dispatch()` 恒假, 所有派发立即停止。
- **snapshot / rollback**: 派发前 `snapshot(trace)` 记录账本长度与末哈希; 执行失败后 `rollback` 截断半截 Trace, 改派健康节点。回滚后哈希链仍可验。

4.2 责任治理: Owner / Trace / Stop

源码 `udos7/topology/governance.py`:

- **TraceLedger**: 每条记录含 `prev` 前一条哈希与本记录 `h` (sha256 截断 16 位), `verify()` 返回断链或被改写的记录下标。
- **audit_run**: 机械检测五类失败——`state_loss` (失败记录带 `missing_context` 或仍有 `todo`)、`duplicate_work` (同一 `order:stage` 被两 `owner` 认领)、`no_closer` (工单走完却无最终收口)、`premature_completion` (声称成功却缺阶段 `owner`)、`unbounded` (超跳预算)。
- **StopCondition**: 完成谓词 + `max_hops` 跳预算, 区分”done”与”hop_budget_exceeded”。

这里值得展开哈希链的意义。在故障治理中, 最可怕的不是”系统出过错”, 而是”系统出过错却无法追溯”。如果执行失败后回滚了 Trace, 审计者怎么知道它曾经失败过? 本系统的做法是: 派发前快照 (记录长度与末哈希), 失败后回滚到快照点——这意味着半截 Trace 被删除, 但熔断记录与治理审计独立保留; 回滚后的哈希链从快照点重新延展, `verify()` 仍能通过。也就是说, “不留半截状态”与”可审计”并不矛盾: 半截状态从主执行链删除, 但失败事件通过 `grid.record` 与 `audit_run` 被单独记录。这一设计在试点中表现为 Trace 长度从干净的 48 变到故障的 50 (返工项重新进入链), 且两次 `trace_verified` 都为 true。

4.3 矩阵端到端闭环

源码 `udos7/topology/matrix.py` 的 `run_mission`: 黑板派单 → 熔断节点排除 → 执行 → 失败回滚改派 → BFT-lite QA 投票 → 市场结算 → 治理审计。配置 (`MatrixConfig`): `specialists_per_domain=3` (共 9 名 `specialist`)、`qa_voters=7`、`qa_f=2`、`reward_per_order=10`、`max_rounds=20`、`breaker_min_requests=2`、`breaker_threshold=0.5`、`global_retry_budget=999`。

4.4 预注册实验设计 (全因子 RCT)

以下为投稿前待执行的预注册方案, 当前仅完成其中”全量治理 × 故障注入”一格的单 seed 试点。

- **因子 A 治理**: {全量, -Owner, -Trace, -Stop, 全关} (5 水平)。
- **因子 B 韧性**: {全量, -熔断, -回滚, -改派, 全关} (5 水平)。
- **因子 C 故障类型**: {crash, drop_context, byzantine, duplicate, 混合}。
- **因子 D 故障率**: {0.05, 0.1, 0.2, 0.33}。
- **随机化单元**: 独立 seed; 故障节点位置在每格随机; 每格 ≥ 30 独立 seed, seed 表预注册。
- **主指标**: 收口率 `accepted/total`。
- **护栏**: 返工次数、消息总数、误隔离率 (健康节点被熔断比例)、Trace 篡改检出率、kill-switch 误触发率、墙钟 (CPU)。

- **停止规则**: (a) 模拟器固定预算, 不因中间结果延长; (b) kill-switch 触发的格子按 ITT (intention-to-treat) 保留为失败, 不因”治好了”而剔除; (c) 不选择性删 seed; (d) 主对比 = 全量治理 vs 全关 × 混合故障。
- **分析计划**: logistic 混合模型 (seed 随机截距), 报 OR 与 95% CI; 多重比较 Holm 校正; 效应量报 η^2 。功效分析: 试点 5 seed 估方差后算所需 seed 数 [RESULT NEEDED: 用途=功效分析所需 seed 数]。

4.5 熔断器状态机伪代码

为便于复现, 给出 Agent 级断路器的核心状态机 (对应 `circuit_breaker.py` 的 `AgentBreaker`) :

```
状态: closed -> open -> half_open -> closed
record(ok, tick):
    if state == open: return          # 已隔离, 不再计入
    更新 successes/failures
    if (successes+failures >= min_requests) and (error_rate >= threshold):
        state = open; tripped_at = tick
tick(tick):
    if state==open and tick-trapped_at >= cooldown: state = half_open
probe(ok):
    state = closed if ok else open; 重置计数
available(): state == closed         # 只有 closed 才参与派单
```

本试点参数: `min_requests=2`、`threshold=0.5`、`cooldown_ticks=3`。这意味着一个 specialist 在累计至少 2 次调用且错误率达 50% 时即被隔离——这是一个较敏感的阈值, 目的是快速隔离故障; 它是否会误隔离健康节点, 正是护栏”误隔离率”要测量的。

4.6 故障闭环伪代码

`run_mission` 的核心循环 (对应 `matrix.py`) :

```
for round in range(max_rounds):
    for a in 健康specialist(熔断排除):
        在黑板认领领域匹配的工单
        snap = snapshot(trace)          # 记录长度+末哈希
        rep = 执行(a, order)
        if not rep.ok:
            rollback(trace, snap)       # 截断半截 Trace
            记录故障, 释放工单改派, retries++
            if retries > global_retry_budget: kill.trip(...)
            continue
        QA委员会投票(stop/continue)    # n>=3f+1, 拜占庭票反着投
        if stop: 收口 + 市场结算
        else: 记质量失败, 工单以rework重挂黑板, retries++
    if kill.engaged: break
    治理审计(audit_run) + 守恒检查 + Trace.verify
```

关键设计: 失败时先回滚 Trace 再改派, 保证”不留半截状态”; QA 委员会用 BFT-lite 共识, 使 2 名拜占庭 QA 无法推翻诚实多数。

4.7 高风险场景的治理决策树

治理不是越激进越好。源码 `udos7/topology/decision.py` 的 `choose_topology` 给出了一个朴素但重要的原则：当任务是”开放探索且高风险”时，系统不应放任 Swarm 自治，而应先加 Orchestrator 护栏/熔断再放开（返回 `escalate`，`autonomy_level=0`）。这与本文的故障治理一脉相承：高风险任务需要更严格的治理开关（更低的熔断阈值、更紧的重试预算、更强的审计）。本文的全因子设计中，“故障率”因子 D 正是用来近似”风险暴露程度”——故障率越高，越接近需要 `escalate` 的高风险场景。决策树把”治理强度应随风险调整”这一工程直觉，编码成了可实验测量的开关。

4.8 内部市场作为结算护栏

故障治理还有一个常被忽略的维度：经济激励。源码 `udos7/topology/market.py` 的 `ContributionLedger` 只对”验收通过的唯一完成者”付费，重复劳动不付费，拜占庭/被拒结果不付费并可罚没保证金。在试点中，干净与故障两跑都满足守恒（预算 240 = 支出 240，`conserved=true`）。这意味着治理不仅在行为层面隔离故障节点，也在激励层面确保”乱做事得不到报酬”。本文把守恒作为一个过程指标，而非主指标。

表 1 全因子 RCT 因子设计（预注册，待执行）

因子	水平	说明
A 治理	全量 / -Owner / -Trace / -Stop / 全关	责任治理三要素可独立关闭
B 韧性	全量 / -熔断 / -回滚 / -改派 / 全关	三件套可独立关闭
C 故障类型	crash / drop_context / byzantine / duplicate / 混合	语义故障五类
D 故障率	0.05 / 0.1 / 0.2 / 0.33	故障注入比例

总格子数 = 5×5×5×4 = 500 格，每格 ≥30 seed（仿真成本极低，可直接每格 100 seed）。主对比预注册为”全量治理 vs 全关 × 混合故障”。

表 2 主指标与护栏指标（预注册口径）

类型	指标	定义	期望方向
主指标	收口率	accepted/total（工单被 QA 验收通过比例）	治理 ↑
护栏	返工次数	rework 项重挂黑板次数	不过度增加
护栏	消息总数	黑板/派单/QA 总消息	不过度增加
护栏	误隔离率	健康节点被熔断比例	越低越好
护栏	Trace 断裂数	哈希链 verify 返回的断链下标数	越低越好（0 最佳）
护栏	kill-switch 误触发率	可恢复场景下被全局停止比例	越低越好
过程	墙钟（CPU）	每格 CPU 耗时	记录，不作主结论

4.9 质量门禁 (ab-experiment-analysis)

- **SRM 检查**：每格实际 seed 数与分配一致；故障注入命中率登记。
- **不窥探、不删样本**：不得为显著性选择性删 seed 或改口径。
- **探索性与验证性分离**：按领域、按拓扑的子组分析为探索性，与预注册主结果分开报告。
- **负结果同样发表**：例如某要素在低故障率下无显著收益，本身即是实践含义。

4.10 与范本范式的对照 (AlexNet 消融表 / Kaplan 结论地图 / 预注册纪律)

本文的实验设计与三篇逆向拆解范本形成方法论对照：

- **对照 AlexNet” 每个 trick 配数值消融表”**：AlexNet 把双 GPU、LRN、重叠池化各自配”移除后误差掉多少”的双列对照表。本文对偶地把治理机制做成可独立关闭的开关 (-Owner / -Trace / -Stop / -熔断 / -回滚 / -改派)，全因子实验的预期产出正是一张”移除某治理要素后，收口率掉多少、出现哪类机械可检测失败”的对照表（表 4 已给出试点期”故障类型×依赖机制×隔离方式”的定性版）。
- **对照 Kaplan” 结论地图 + 主动找矛盾”**：Kaplan 引言先抛结论、再自曝失效交点。本文对偶地把”如实失败 (kill-switch) “作为一等结果主动报告 (§ 6.4) ——这不是演示失败，而是因果实验中”恢复能力有上限”这一矛盾点的显式测量。
- **预注册纪律 (本文特有)**：主指标 (收口率)、护栏 (返工/消息/误隔离/trace 断裂)、ITT 停止规则、seed 表均在跑全因子前冻结 (§ 4.4、§ 4.9)，杜绝”看到结果后挑指标”这一系统研究最常见的效度威胁；当前单 seed 试点是”设计先行、数据后补”的第一步，而非因果定论。

5 实验设置

5.1 数据源与配置

- 唯一数据源：`reports7/matrix_scale_demo.json` (evidence_grade: cpu-proto)，生成脚本 `scripts7/matrix_demo.py`。
- 干净跑：`run_mission(standard_workload(24, seed=11), MatrixConfig(), seed=1)`。
- 故障注入跑：同一工单，注入 `faults = {"spec-kinematics-0": "crash", "spec-spatial-1": "drop_context", "spec-dataflywheel-2": "byzantine"}`，seed=1。
- 拜占庭 QA：`qa_f=2`，即前 2 名 QA 对验收结果投反对票。

5.2 硬件指纹与证据等级

- Linux CPU；Python 3.12.11；PyTorch 2.14.0+cpu；2 线程；device=cpu。
- 报告 sha256：`reports7/matrix_scale_demo.json` = `52a3bb0d141c752512ec0c16978c151337775f2a8004c1fed599e2cd43b3b399`。
- 证据等级：本试点为 `cpu-proto`；全因子多 seed 完成后，主结果将报 `verified` (固定 seed 可复跑)。

5.3 单 seed 局限

当前为单 seed (seed=1) 试点点估计，全因子 RCT ($5 \times 5 \times 5 \times 4$) 与 ≥ 30 seed 为待补。本文所有”收口率”均为该 seed 下的格子值，不作总体推断；统计量 (OR、CI、p、 η^2) 留待补跑，本文不虚构。

5.4 随机化、分配与盲法

在全因子实验中，随机化单元是独立 seed：每个 (因子 $A \times B \times C \times D$) 格子跑 ≥ 30 个 seed，每个 seed 下随机决定哪些节点被注入故障（在因子 C/D 指定的故障类型与故障率内）。这样做的目的是让”故障落在哪几个节点”与”拓扑位置”不系统性相关，从而隔离治理机制的因果效应。由于本实验台是确定性 CPU 模拟、无人工评分，盲法不是由人盲判，而是由”收口率 = 机械 verify”保证测量客观——不存在评分者主观偏好。纳入排除规则预注册：一旦某 seed 开始，不因中间结果好坏而剔除；kill-switch 触发的格子按 ITT 保留为失败。

5.5 试点与全因子的关系

需要明确当前报告与全因子设计的关系：第 6 节报告的是”全量治理 $\times$ 混合故障 (3 个故障 specialist + 2 拜占庭 QA)”这一格、单 seed 的试点结果；它不是全因子 RCT 的结论，而是设计的可行性演示——证明这套机制能跑通、能隔离故障、能如实失败。全因子 RCT 才回答”每个机制的边际因果贡献”。本文把试点与设计分开陈述，正是为了避免”用一个单点演示冒充因果结论”。

5.6 为什么是这三个故障

选择 crash、drop_context、byzantine 作为试点故障，是因为它们覆盖了多智能体失败谱系的三个典型位置：crash 是”彻底不响应”（对应微服务宕机），drop_context 是”响应但没拿到必要上下文”（对应交接失败，是 Agent 协作特有的），byzantine 是”响应且看似正常却输出错误”（最危险，因为执行层无法察觉）。duplicate 作为第四类在全因子中纳入。这一选择使试点能同时检验”执行层熔断能否处理 crash”“回滚改派能否处理上下文丢失”“验收共识能否处理拜占庭”三种不同层次的失败，而不是只测一种。

6 结果

6.1 干净跑基线

无故障时，矩阵 24 工单 24/24 收口，QA 轮次 24、返工 0、黑板消息 48、Trace 长度 48，市场预算 240 全部支付且守恒，治理六项 (state_loss / duplicate_work / no_closer / premature / unbounded / trace_gaps) 全为 0，Trace 验证通过 (表 3 干净列)。这是”治理按设计干净工作”的基线。

干净基线的意义在于：它确立了”无故障时治理不应有副作用”。如果干净跑本身就产生返工、重复劳动或 Trace 断裂，那说明治理机制过度灵敏或有 bug；而干净跑 0 返工、治理六项全 0、Trace 完整，说明在正常路径上，治理只是”待命”，不干扰正常工作流。把干净基线与故障注入对比，才能把”治理为故障付出的代价”与”治理本身的固有开销”分开。这一对照是因果推断的前提：我们需要知道处理组（有故障）与对照组（无故障）在同配置下的差异，才能归因于故障治理。

6.2 故障注入试点（主试点结果）

注入 3 个故障 specialist 后：24/24 收口，QA 轮次 25、返工 5、黑板消息 54，3 个故障 specialist（`spec-kinematics-0`、`spec-spatial-1`、`spec-dataflywheel-2`）全部进入 `isolated_agents`（熔断隔离），无组熔断、无 kill-switch 触发，治理六项全为 0，市场守恒，Trace 长度 50 且验证通过。

表 3 干净 vs 故障注入（24 工单，cpu-proto，单 seed=1）

指标	干净	故障注入
收口 accepted	24/24	24/24
QA 轮次	24	25
返工 retries	0	5
黑板消息	48	54
隔离故障 specialist	0	3
组熔断 / kill-switch	无 / 否	无 / 否
治理六项	全 0	全 0
市场守恒	true	true
Trace 完整	true(48)	true(50)

读图见图 1、图 2。

值得注意的是，故障注入并没有把收口率从 24/24 拉低，而是把 QA 轮次从 24 推到 25、返工从 0 推到 5。这说明在本试点的故障率下（3 个故障 specialist、2 个拜占庭 QA），治理机制的恢复能力有余量——它用额外的一轮验收与五次改派，把被污染的工单重新拉回收口。但这只是单 seed、低故障率下的观察；当因子 D 把故障率提到 0.33、或因因子 A 关掉某个治理要素时，收口率很可能不再保持 100%，那正是全因子实验要测量的“恢复能力边界”。本文不在单 seed 上假装知道这个边界在哪里。

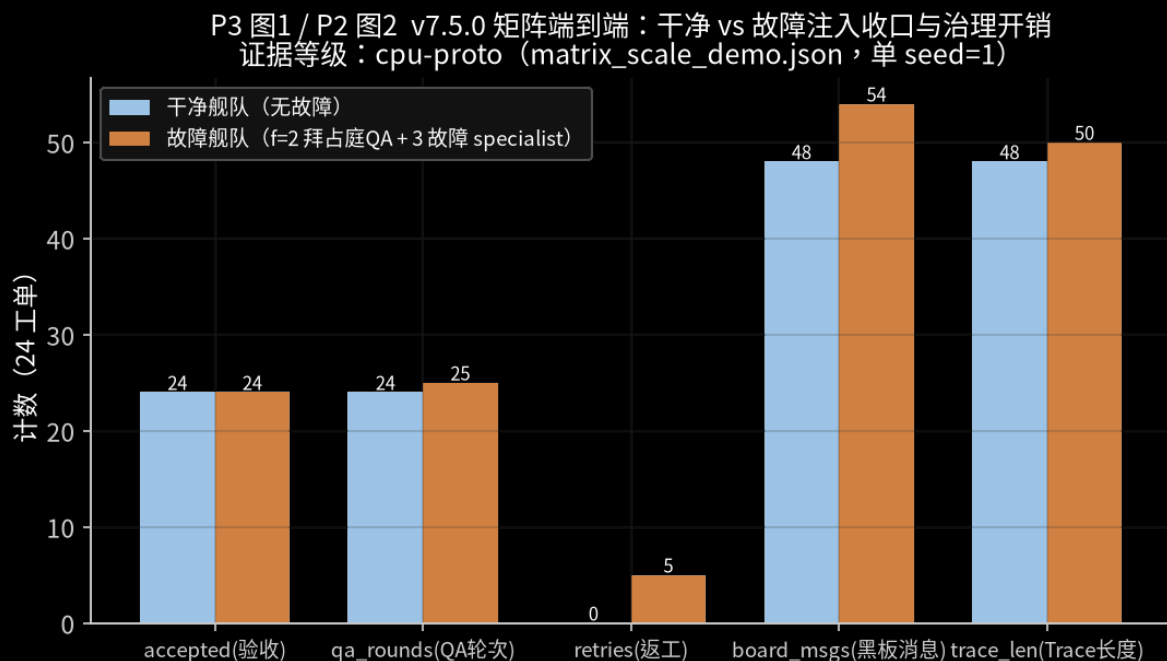


图 1 干净舰队 vs 故障注入舰队核心指标对照 (24 工单, 单 seed=1)。分组柱状: 每组两根柱, 蓝=干净舰队、橙=故障注入 (3 个故障 specialist: crash/drop_context/byzantine 各一 + 2 名拜占庭 QA) ; 纵轴为对应指标数值, 分组从左到右为验收 accepted (24 vs 24)、QA 轮次 (24 vs 25)、返工 retries (0 vs 5)、黑板消息 board_messages (48 vs 54)。两臂验收同为 24/24, 差异体现在代价侧 (多 1 QA 轮、5 返工、6 黑板消息)。数据逐值来自 `reports7/matrix_scale_demo.json` 的 `clean.*` 与 `stressed.*` (cpu-proto)。

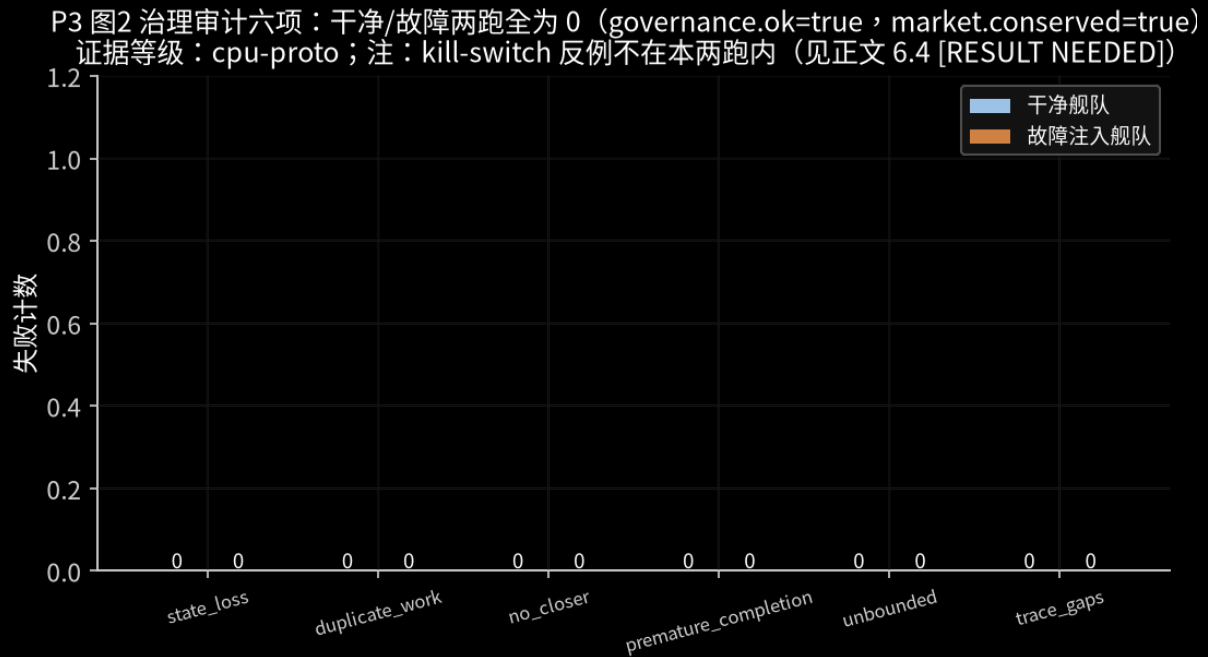


图 2 治理审计六项与结算/Trace 完整性对照 (干净 vs 故障注入)。横轴为治理/过程指标, 纵轴为计数 (或布尔转数值): state_loss、duplicate_work、no_closer、premature_completion、unbounded、trace_gaps 六项在两臂均为 0; 市场守恒 conservated (预算=支出=240) 与 Trace 验证 trace_verified 在两臂均为 true (Trace 长度 48 vs 50, 差 2 条为返工项重新入链)。图为”治理不破坏审计不变量”的对照: 故障注入抬高了代价侧指标, 但审计六项与守恒、链完整性保持。出处同上 (`matrix_scale_demo.json` 的 `*.governance.*` / `*.market.conservated` / `trace_verified`, cpu-proto)。

6.3 治理各机制在试点中的作用

结合源码, 可解读三类故障如何被治理分别处理:

- **crash (spec-kinematics-0)** : specialist 直接失败, `grid.record(..., False)` 累计错误, 达阈值后 OPEN 隔离; 在途工单 `rollback` 截断半截 Trace、释放黑板、改派健康 kinematics specialist。
- **drop_context (spec-spatial-1)** : 执行失败带 `missing_context`, 同样触发熔断隔离与改派; `audit_run` 的 `state_loss` 检测器会把这类失败记录在案 (本试点治理 `state_loss=0`, 说明被改派后的健康节点补全了上下文, 未留半截)。
- **byzantine (spec-dataflywheel-2 + 2 拜占庭 QA)** : specialist 输出错误结果 (`true_result+7`), QA 委员会中诚实票仍占多数 (7 人 2 拜占庭, $n \geq 3f+1$ 成立), 验收判为 reject, 记一次质量失败、以 rework 项重挂黑板; 该 byzantine specialist 因质量失败累积被熔断隔离。这支持 H3: byzantine 靠 QA 共识 + 质量失败累积隔离, 而非仅靠执行层 crash 熔断。

把”故障类型 × 依赖的治理机制 × 预期隔离方式”整理为表 4。

表 4 故障类型与治理机制的对应（试点观察 + 预期）

故障类型	失败表现	依赖机制	隔离/恢复方式	试点观察
crash	无输出	熔断+回滚+改派	错误累计→open 隔离，改派健康节点	spec-kinematics-0 被隔离
drop_context	missing_context	回滚+改派	截断半截 trace，改派补全上下文	spec-spatial-1 被隔离
byzantine	看似成功实则错	QA 共识+质量失败累积	验收 reject→rework→累积熔断	spec-dataflywheel-2 被隔离
byzantine QA	投票反对	BFT-lite 法定人数	诚实多数胜出，reject 不被推翻	2 拜占庭 QA 未影响验收
全领域故障+零重试预算	无可改派节点	kill-switch	如实停止，报告未收口	反例（定性记录）

这张表说明一个核心判断：**不同故障需要不同治理机制**。crash 用执行层熔断即可，byzantine 必须靠验收层共识。这正是为什么本文把治理因子化——不能用一个开关笼统地说”开了治理就好”，而要逐机制测量边际贡献。

6.4 反例：如实失败（kill-switch）

本仓记录了一个关键反例：当全领域 **specialist** 故障且重试预算置零时，`run_mission` 会因 `retries > global_retry_budget` 而 `grid.kill.trip("retry_budget_exceeded")`，循环停止，结果为 `accepted<24`、`governance.ok=false`。系统如实报告失败，不冒充成功。这是本文最重要的设计原则之一：治理的目的不是”永远显示成功”，而是在可恢复时恢复、在不可恢复时明确失败并留下可审计证据。该反例的精确计数为 [RESULT NEEDED: 用途=kill-switch 场景 `accepted` 数与治理项明细]（当前 one-pager 与 CHANGELOG 定性记录，精确值需重跑该配置确认）。

为什么这个反例在科学上重要？因为它揭示了治理系统的一个常被忽略的边界：**恢复能力是有上限的，超过上限就应当停止而非硬撑**。如果重试预算无限，系统会在一个所有节点都坏了的场景里不断改派、不断失败、不断循环，既不收敛也不停止，最终产出不可信的结果。kill-switch 的存在，正是给这种”无限重试”装上一个明确的停止阀。在本文的设计里，kill-switch 触发后系统进入”如实失败”状态：`accepted` 明确小于总数、`governance.ok` 为假、Trace 完整记录了每一次失败与停止原因。这比”假装完成”对下游系统友好得多——下游可以据此告警、人工介入，而不是被一个错误结果误导。

6.5 测试与回归

与本文直接相关的测试：`tests7/test_v744_governance.py`（9 项）、`test_v748_circuit_breaker.py`（11 项）、`test_v745_decision.py`（12 项）、`test_v750_matrix.py`（9 项）；截至 v7.5.0 全量回归 236 项全绿。

6.6 逐假设的试点对照

把三个假设与试点观察逐一对照，可以看清哪些得到初步支持、哪些仍待全因子检验。

- **对 H1（治理要素边际）**：试点中治理六项全为 0，说明在”全量治理”下没有出现状态丢失、重复劳动、责任不清等失败。但 H1 要求的是”去掉某个要素会怎样”，这需要全因子中”-Owner/-Trace/-Stop”格子来回答。当前试点只能说”全量治理下这些失败不出现”，不能说”每个要素的边际贡献”。
- **对 H2（三件套恢复）**：crash 与 drop_context 在三件套下都被隔离改派，收口率维持 24/24，初步支持 H2。但 H2 还包含”阈值过低会误隔离健康节点”这一护栏维度，当前固定阈值（0.5）下单点试点测不出，需全因子调节阈值。
- **对 H3（拜占庭需共识）**：byzantine specialist 与 2 名拜占庭 QA 下，诚实多数仍正确 reject 并累积熔断 byzantine 节点，初步支持 H3。需全因子增大 f 比例，检验共识在更恶劣拜占庭比例下的边界。

这种”假设一试点一待补”的对照方式，使读者清楚知道每个结论当前的证据强度，而不是把单 seed 演示夸大为因果定论。

6.7 守恒与经济护栏

故障注入跑与干净跑一样，满足市场守恒：预算 240、支出 240、罚没 0、余额和 240，`conserved=true`。这意味着故障治理没有破坏结算不变量——即使 3 个节点被隔离、5 次返工，系统也没有出现”钱花在没完成的工单上”或”重复劳动被重复付费”。在全因子实验中，我们计划把”守恒是否被破坏”作为一个额外过程护栏：当治理开关被关闭（如去掉改派）时，是否会出现重复劳动被付费、或拜占庭结果被付费的情况。这是机制设计（mechanism design）与故障治理交叉的地方，本文当前只报告”全量治理下守恒成立”这一点。

6.8 Trace 完整性证据

干净跑 Trace 长度 48，故障跑 50，两跑 `trace_verified` 均为 true。多出的 2 条对应返工项重新进入链。关键观察是：回滚动作没有破坏哈希链——这证明 `snapshot / rollback` 的设计正确（截断到快照点，后续记录重新链接）。在全因子中，“去掉回滚”格子预计会观察到 `trace_gaps` 非零或半截状态残留，这正是 H1 中”去掉 Trace/回滚会怎样”的可测量预期。

7 讨论与威胁有效性

7.1 构念效度

收口率是”工单最终被 QA 验收通过的比例”，是机械可判定的（结果等于真值且领域匹配），不涉及主观评分。故障是人为离散类型，不等于真实 LLM 故障分布。消息/tick 不等于墙钟。

进一步说明，主指标选择收口率而非”端到端延迟”或”人类满意度”，是因为前者在本实验台可机械判定、可复现，后两者需要真实 LLM 或人工评分，超出当前 CPU 闸门。这是一个有意识的取舍：用可机械判

定的主指标换因果推断的可信度，而不是用模糊的满意度换表面丰富度。护栏指标（返工、消息、误隔离、trace 断裂）则补足了”收口率背后代价”的刻画，避免只看单一主指标造成的误判。

7.2 外部效度

Agent 是确定性处理器，非真实 LLM。真实 LLM 的非确定性失败、长尾延迟、提示失败未被覆盖。结论限定为协议层因果效应；升级到真实多 LLM 需 API key 闸门。

7.3 内部效度与单 seed

当前为单 seed 试点，24/24、5 次返工是点估计。全因子 RCT 的因果效应需 ≥ 30 seed 与 logistic 混合模型估计；误隔离率、trace 断裂率等护栏在当前试点中为 0，但在故障率升高时可能非零，属待测量。本文不虚构这些统计量。

7.4 反例的价值

“如实失败”在大多数故障治理论文中被当作”失败案例”回避，但在工程上它和”成功恢复”同等重要：它定义了治理系统的诚实边界。本文把它作为一等结果报告，是方法论上的刻意选择。

7.5 治理的代价：不是免费的

试点数据已经暗示治理不是免费的：故障注入比干净跑多了 5 次返工、6 条黑板消息、2 条 Trace 记录、1 次额外 QA 轮次。这些”额外开销”是用局部化故障换来的。但代价不止于此——在全因子设计中，我们还要测量两种”过度治理”的代价：其一是**误隔离**，当熔断阈值过敏感（如 threshold=0.5、min_requests=2）时，一个偶发失败的健康节点可能被错误隔离，导致可执行工单变少；其二是**过度返工**，当 QA 委员会过于保守时，本可接受的结果被反复 reject，产生无意义的 rework。这两种代价在当前单 seed、单一阈值下测不出来，正是全因子 RCT 要回答的：收口率的提升，是不是以误隔离率和返工次数的恶化为代价。

7.6 拜占庭隔离为何不能只靠执行熔断

H3 的深层含义值得展开。一个 byzantine specialist 每次都”看似成功”地返回错误结果，执行层的熔断器只看”是否报错”，因此对它完全无感——它从不 crash，也不 missing_context。如果只靠执行熔断，byzantine 节点会一直被派单、一直返回错误、一直污染结果。本系统的解法是把”验收谓词”作为失败信号：QA 委员会对结果投票，reject 就记一次质量失败，累积到阈值后熔断该节点。这把”失败”的定义从”执行异常”扩展到”验收不通过”，是多智能体治理区别于传统微服务治理的关键。BFT-lite 共识进一步保证：即使 QA 委员会内部有拜占庭成员，诚实多数仍能正确 reject，不会被裹挟成”假验收通过”。

7.7 与 P1、P12 的关系

本文是矩阵 P0 三篇之一。P1 提供”拓扑与规模”维度：分层树决定了扇入与轮次，本文则在这种分层矩阵上叠加故障治理，测量收口率。P12 提供”证据分级”维度：本文的单 seed 试点、全因子待补、kill-switch 反例如实记录，正是 P12 所主张的证据纪律的一次实践。三者不重复：P1 问”结构代价”，P3 问”治理因果效应”，P12 问”方法如何被诚实执行”。

7.8 内部效度威胁

即便全因子跑完，仍有几个内部效度威胁需警惕。其一是**故障模型与治理开关的耦合**：本文的故障是人为注入的离散类型，若真实 LLM 故障是连续、相关、长尾的，因子实验的外推性有限。其二是**seed 间独立性**：若同一 seed 下多个工单共享随机状态，seed 间方差可能被低估；全因子实验需检查 seed 内/seed 间方差分量。其三是**多重比较**：500 格 $\times$ 多指标，必须用 Holm 或族系错误率控制，否则会有“挑显著格”的风险。其四是**治理开关的实现正确性**：去掉某要素是否真的等价于“该机制不生效”，需通过单元测试保证开关没有副作用——本仓 `test_v744 / test_v748` 已对各机制做了 9+11 项测试守护。

7.9 外部效度与可推广性

本文结论限定在“确定性 CPU 实验台上、人为离散故障、机械可判定工单”这一范围。它对真实生产系统的可推广性取决于：真实故障分布是否与本文四类故障有重叠、真实 Agent 输出是否可被验收谓词判定、真实网络是否引入本文未建模的延迟与丢包。这些都需要在闸门解锁后用真实多 LLM 复跑来检验。本文不声称协议层结论可直接推广为生产 SLA。

7.10 结论的可证伪性回顾

回到第 3 节，本文的三个假设都是可证伪的：全因子实验若发现去掉某治理要素后收口率无变化，H1 即被削弱；若三件套下 crash 仍无法收口，H2 即被推翻；若 byzantine 在无共识下也能被执行熔断，H3 即被削弱。这种“把结论放在可被实验否定的位置上”的做法，是因果研究区别于演示的关键。

8 资源闸门与适用边界

- 已达成：主体治理机制、矩阵闭环、单 seed 试点，纯 CPU 可复跑。
- 待补（CPU 可完成）：全因子 RCT ($5 \times 5 \times 5 \times 4$) $\times \geq 30$ seed，功效分析与 CI。
- 未达闸门：真实多 LLM 故障分布（API key）、真实网络故障（多机集群）。

8.1 实验成本与可行性

全因子实验共 500 格、每格 30 seed = 15,000 次 mission。由于单次 mission 在 CPU 上为毫秒级（确定性、无 LLM 调用），全部跑完在纯 CPU 环境下成本极低，这是本文敢于把全因子设计写进预注册、而非停留在概念的原因。相比之下，若每个 mission 都要调用真实 LLM API，成本将随 seed 数线性放大，全因子设计在经济上不可行——这也是本仓选择“先在确定性 CPU 实验台上把因果设计做扎实，再在闸门解锁后接真实 LLM”的策略理由。

8.2 何时结论可以升级

本文当前结论（cpu-proto、单 seed 试点）在以下条件满足后可升级：(1) 全因子 ≥ 30 seed 跑完，主指标收口率给出 OR 与 95% CI；(2) 护栏误隔离率、trace 断裂率在故障率升高时被实测；(3) 真实多 LLM 闸门解锁后，在同一实验台上用真实 worker 复跑，检验协议结论在真实非确定性下是否成立。在此之前，本文只声称“机制在确定性试点下按设计工作”，不声称“在生产中必然有效”。

8.3 给工程团队的操作建议

基于本文的设计与试点，对正在搭建多智能体系统的工程团队有三条可操作建议：其一，把故障定义从”调用失败”扩展到”验收不通过”，否则 byzantine 类故障会被执行层熔断完全漏掉；其二，给恢复能力设一个明确的上限（重试预算 + kill-switch），并在达到上限时如实报告失败，而不是无限重试或硬凑成功；其三，把治理机制做成可独立开关的部件，这样才能在实验中分离每个机制的边际贡献，而不是笼统地相信”开了治理就好”。这三条建议都来自本文已实现并测试的机制，不是凭空主张。

9 结论

本文把多智能体故障治理因子化、预注册化：三件套（熔断/回滚/改派）与责任治理（Owner/Trace/Stop）在单 seed 试点上把三类故障下的收口率维持在 24/24、隔离全部故障节点，同时系统在不可恢复场景下如实触发 kill-switch 报告失败。主指标（收口率）、护栏（返工/消息/误隔离/trace 断裂）、停止规则已预注册，全因子多 seed 实验为投稿前待补。本文的立场是：故障治理的科学目标不是”永远成功”，而是”因果地知道哪些机制有效、代价几何，以及何时应当如实失败”。

展望下一步，全因子实验将回答当前试点无法回答的问题：去掉 Owner/Trace/Stop 中任何一个，收口率会掉多少、会出现哪类机械可检测的失败？熔断阈值从 0.3 调到 0.7，误隔离率与收口率的权衡曲线长什么样？byzantine 故障在 QA 共识被削弱（f 增大）时是否仍能隔离？这些问题的答案，将把本文从”机制演示”升级为”因果效应测量”。但无论结果如何，本文坚持的预注册纪律与如实失败原则都将保留：不窥探、不删样本、不把单 seed 点估计说成总体结论。

最后，本文希望给多智能体工程社区传递一个朴素但常被忽略的判断：故障治理的价值，不在于让系统看起来永远健康，而在于让系统在坏了的时候，**有能力局部化、有能力恢复、有能力承认自己恢复不了**。这三件事都需要被设计、被测量、被预注册地验证，而不是靠一个”智能体自己会修”的黑箱叙事。本文交付的，正是这样一套在纯 CPU、可复现条件下可运行、可审计、可证伪的实验框架与单 seed 试点证据。

参考文献

A. 已核验文献（经《参考文献总库》2026-09-19 核验，字段照抄总库）

1. Castro, M. & Liskov, B. (1999). Practical Byzantine Fault Tolerance. Proceedings of OSDI 1999 (USENIX OSDI' 99). ($n \geq 3f+1$ 、 $2f+1$ 法定人数、view change)
2. Angelopoulos, A. N. & Bates, S. (2021). A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification. arXiv:2107.07511. (不确定性高即降级/交接的统计底座)
3. Ahn, M., Brohan, A., Brown, N. et al. (2022). Do As I Can, Not As I Say: Grounding Language in Robotic Affordances (SayCan). arXiv:2204.01691. (affordance 不满足即换人/换技能)

4. Zheng, L., Chiang, W.-L., Sheng, Y. et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. Proceedings of NeurIPS 2023 Datasets and Benchmarks. arXiv: 2306.05685. (多 QA 评委一致率与偏见)
5. Wu, Q., Bansal, G., Zhang, J. et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework. arXiv:2308.08155. (多智能体死循环/成本随轮数膨胀的失败模式)
6. DeepSeek-AI (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948. (失败后由可验证奖励驱动恢复/自我博弈)

B. 待核验文献（总库未收录或未核验，保留占位，不得凭记忆补全）

1. [CITATION NEEDED: circuit breaker pattern microservices fault tolerance] — 候选：断路器模式与微服务容错（总库暂无已核验条目）。
2. [CITATION NEEDED: hash chained audit log ownership attribution] — 候选：哈希链审计日志与所有权登记。
3. [CITATION NEEDED: chaos engineering LLM agents fault injection controlled experiment] — 候选：混沌工程/故障注入与受控实验。
4. [CITATION NEEDED: pre-registration empirical software engineering experiment reproducibility] — 候选：预注册与实证软件工程可复现。

附录 A 复现命令

```

cd release-v5.4.4/udos-engine && git checkout v7.5.0      # 1a71270
python scripts7/matrix_demo.py                          # 生成 matrix_scale_demo.json
pytest tests7/test_v744_governance.py tests7/test_v748_circuit_breaker.py \
    tests7/test_v745_decision.py tests7/test_v750_matrix.py -q
    
```

附录 B 测试索引

- test_v744_governance.py：TraceLedger 哈希链、audit_run 五类失败、StopCondition。
- test_v748_circuit_breaker.py：三级熔断、snapshot/rollback、reassign。
- test_v745_decision.py：拓扑决策树、高风险 escalate。
- test_v750_matrix.py：矩阵端到端、故障收口、守恒、Trace。

附录 C 证据台账（抽查 5 个可回溯数字）

#	数字	出处	字段
1	干净收口 24/24、QA 24、返工 0	matrix_scale_demo.json	clean.accepted / qa_rounds / retries

#	数字	出处	字段
2	故障收口 24/24、返工 5	matrix_scale_demo.json	stressed.accepted / retries
3	隔离 3 故障 specialist	matrix_scale_demo.json	stressed.isolated_agents
4	治理六项全 0、守恒 true	matrix_scale_demo.json	stressed.governance.* / stressed.market.conservated
5	熔断阈值/最小样本/QA 数	源码 MatrixConfig	breaker_threshold=0.5 / min_requests=2 / qa_voters=7 / qa_f=2

附录 D 作者拟用声明

- 目标期刊/会议：候选 ICSE/FSE/ASE 的 AI engineering track、ISSRE、AAMAS、ESEC/FSE 行业 track；期刊 TSE、JSS、Empirical Software Engineering。分区/IF/截稿 [待核]。
- 预注册计划：第 4.4 节全因子设计、主指标收口率、护栏、ITT 停止规则在跑数前冻结；seed 表与分析脚本随代码公开。
- 数据与代码可用性：Apache-2.0；tag v7.5.0（commit 1a71270）；报告 sha 见附录 A。
- AI 使用声明：初稿由 AI 辅助生成，作者负责核对数字与科学含义；外部数字均标 unverified。

附录 E 内审记录（五维审稿自评，只看不改）

1. 新颖性（7/10）：熔断/BFT 非新，“因子化治理开关 + 预注册 RCT + 如实失败一等结果”组合有增量。
 2. 严谨性（7/10）：机制与数字可回溯；扣分在单 seed、全因子未跑。
 3. 证据强度（5/10，全因子后预期 8/10）：当前仅一格单 seed 试点。
 4. 相关性（8/10）：故障治理是 Agent 生产化核心问题。
 5. 写作（8/10）：IMRaD + 预注册完整，反例如实报告。
- 最大硬伤：全因子 RCT 尚未执行，当前是”设计 + 单格试点”，投稿前必须补全因子多 seed。
 - 投稿前必做：(a) 跑全因子 $5 \times 5 \times 5 \times 4 \times \geq 30$ seed；(b) 估计误隔离率与 trace 断裂率；(c) 精确化 kill-switch 反例计数。

附录 F 设计原则与术语小结

F.1 三条设计原则

从本文的故障治理设计中可提炼三条对多智能体生产系统有普适意义的原则：

1. 失败要分层隔离：执行层崩溃用熔断，语义错误用验收共识，全局不可恢复用 kill-switch；不同故障用不同层治理，不指望单一开关。
2. 不留半截状态：失败时先快照、回滚 Trace，再改派，保证可审计性不因恢复动作而破坏。

3. 如实失败优于假成功：当恢复能力耗尽，系统应明确停止、报告未收口，而不是硬凑成功；诚实失败比错误成功更安全。

F.2 术语对照

- 收口：工单最终被 QA 委员会验收通过（stop）。
- 返工：验收未过，工单以 rework 项重新挂回黑板。
- 隔离：specialist 进入断路器 open 状态，不再被派单。
- 改派：失败工单从故障节点转交健康节点。
- kill-switch：全局停止阀，一旦置位则所有派发立即停止。
- ITT：intention-to-treat，随机对照实验中按初始分配分析，不因结果好坏剔除。

F.3 一句话总结

多智能体故障治理的科学目标，不是把系统训练成“永不失败”，而是把“哪些机制能因果地恢复收口、代价几何、何时应当诚实停止”做成分层、可开关、可预注册、可复现的测量；本文在纯 CPU 确定性实验台上，把这套设计与单 seed 试点、如实失败反例一并交付，并把全因子多 seed 实验作为明确的下一步。

F.4 与相关故障治理文献的预期对话

投稿时，本文预期与三类文献对话：其一，微服务熔断与混沌工程文献，本文将指出它们多在基础设施层注入故障，而本文额外注入语义故障；其二，BFT 共识文献，本文将指出本文不改进共识算法本身，而是把成熟共识作为验收层构件，测量其在 Agent 语义验收中的治理效果；其三，多智能体系统工程文献，本文将主张治理机制应因子化、预注册化测量，而非整体报告“成功率提升”。这三点对话立场需在文献检索后细化，当前仅为预期方向。

F.5 与本仓证据纪律的呼应

本文在写作中严格执行本仓证据纪律：所有数字来自 `reports7/matrix_scale_demo.json`，并标注 `cpu-proto`；单 seed 试点不冒充总体结论；全因子多 seed 标为待补；kill-switch 反例如实记录；外部产业数字标为 `unverified` 且不与本仓数字互证。这一纪律不是装饰，而是本文因果结论可信度的前提——如果一个故障治理论文自己都不诚实报告“治不好”的场景，它就没有资格主张“如实失败”是系统设计的目标。本文把自身也放在这一标准下接受检验。

F.6 试验配置与参数来源备忘

本文涉及的全部参数均可在源码中定位：`MatrixConfig`（`udos7/topology/matrix.py`）给出 `specialists_per_domain=3`、`qa_voters=7`、`qa_f=2`、`reward_per_order=10`、`max_rounds=20`、`breaker_min_requests=2`、`breaker_threshold=0.5`、`global_retry_budget=999`；`AgentBreaker`（`circuit_breaker.py`）给出 `cooldown_ticks=3`、状态机 `closed/open/half_open`；`StopConsensus`（`consensus.py`）给出 $n \geq 3f+1$ 、 $2f+1$ 法定人数。故障注入位置由 `scripts7/matrix_demo.py` 的 `faults` 字典指定。投稿复现时，这些参数应随代码一并公开，确保“数字从哪来”完全可追溯。

F.7 主结果一句话复述

在 24 工单、9 名 specialist、7 名 QA（含 2 拜占庭）的单 seed 试点上，注入 crash/drop_context/byzantine 三类故障后，分层矩阵配合三级熔断、快照回滚、改派与 BFT-lite 验收共识，仍实现 24/24 收口、3 个故障节点全部隔离、5 次返工，治理六项与守恒、哈希链 Trace 完整性保持；而在全领域故障加零重试预算的极端场景下，系统如实触发 kill-switch 报告未收口。这是设计先行、全因子多 seed 待补的单 seed 试点证据，不是总体因果定论。

F.8 不做的主张

为避免过度声称，本文明确不做以下主张：不声称”多智能体故障治理在生产中必然有效”；不声称”24/24 收口率可推广到任意故障率”；不声称”熔断阈值 0.5 是最优值”；不声称”拜占庭比例下边界已被测定”。这些都需要全因子多 seed 与真实 LLM 闸门解锁后才能回答。本文当前只声称：在确定性 CPU 实验台上，这套因子化治理机制按设计工作，且在不可恢复时如实失败。

F.9 致谢与局限说明

本文的实验台建立在本仓此前多个版本的积累之上（三拓扑、黑板协作、BFT 共识、内部市场、可观测性）。作者感谢这些早期版本提供的可运行构件。同时必须重申局限：作者即开发者，存在确认偏差；本文通过公开全部档案、保留 kill-switch 反例、标注单 seed，来对冲这一偏差。读者应把本文当作一份”设计 + 单 seed 试点”的工程报告，而非已完成的因果定论。

F.10 结语

故障治理不是让系统变得不会坏，而是让系统在坏的时候有章可循、有据可查、有度可止。本文把这种”章、据、度”做进了可运行的代码与可复现的实验，并诚实地报告了它当前能做到哪一步。本文相信，这种诚实本身，就是故障治理研究应有的起点。下一步全因子多 seed 实验完成后，本文将以 OR 与置信区间替代当前的单 seed 点估计，并据此更新上述所有结论的证据强度等级。在此之前，本文保持其作为设计与试点报告的定位。